STUDENT CASE STUDY 3: API DEVELOPMENT & SYNC LOGIC

Course: CMU-CS 445 – *System Integration Practices*
Academic: 1- 2025–2026
Instructor: MSc. Nguyen Dang Quang Huy

1. Objective

The objective of Case Study 3 is to implement a functional backend API layer that integrates the two enterprise systems used by Company X:

- **HUMAN System** – SQL Server database *(HUMAN_2025)*
- **PAYROLL System** – MySQL database *(payroll)*

Using the design artifacts from Case Study 2—including the SRS, Integration Architecture, Data Mapping Document, and API Integration Plan students must develop:

- RESTful APIs enabling bidirectional data exchange
- Synchronization logic for cross-database consistency
- Validation and transaction control
- Error-handling and rollback mechanisms
- Unit tests and integration tests ensuring reliability

This API layer will serve as the backend foundation for the Dashboard developed in Case Study 5.

2. Scenario Background

Company X operates two independent systems:

1. **HUMAN_2025 (SQL Server)**
    - Stores employees, departments, positions, dividends
    - Acts as the master source of truth for HR data
2. **payroll (MySQL)**
    - Stores salary, attendance, and synchronized HR copies
    - Supports financial processing and payroll reporting

Because the CEO requires **full integration without modifying either database**, all synchronization must occur through an external API service that:

- Connects to both DBs simultaneously
- Reads HR data and writes synchronized records to payroll

- Provides unified API endpoints for Dashboard consumption
- Handles atomic updates across heterogeneous databases
- Logs all operations for audit purposes

Case Study 3 turns the CS2 design into **a working backend implementation**.

3. Tasks and Requirements

3.1 Backend API Implementation

Students must implement the integration backend using one of the following:

- **C# (.NET 8 Web API)**
- **Python (Flask or FastAPI)**

The backend must connect to:

- **SQL Server (HUMAN_2025)** using official ODBC/SQLClient
- **MySQL (payroll)** using proper MySQL connectors

Required API Endpoints

(Directly derived from CS2's API Integration Plan)

A. Employee-Related APIs

Method	Endpoint	Description	Source
GET	/employees	Retrieve employees from HUMAN_2025	SQL Server
GET	/employees/{id}	Retrieve a specific employee	SQL Server
POST	/employees	Insert employee into HUMAN_2025 and sync → payroll	SQL Server → MySQL
PUT	/employees/{id}	Update employee in HUMAN_2025 and sync → payroll	SQL Server → MySQL
DELETE	/employees/{id}	Delete employee after constraint validation	SQL Server + MySQL

Special Logic

- Deletion must be blocked if employee exists in:
  - **PAYROLL.salaries**
  - **HUMAN_2025.Dividends**
- Sync must ensure employees_payroll.EmployeeID stays consistent

B. Department & Position APIs

Method	Endpoint	Description
GET	/departments	Get HR departments
POST	/departments	Create a new department and sync
PUT	/departments/{id}	Update department and sync
GET	/positions	Get HR positions
POST	/positions	Add position and sync
PUT	/positions/{id}	Update position and sync

Sync destination

- PAYROLL.departments_payroll
- PAYROLL.positions_payroll

C. Payroll APIs

Method	Endpoint	Description
GET	/payroll	Retrieve all salary records
GET	/payroll/{id}/history	Get salary history for an employee

Data is always retrieved from **PAYROLL.salaries**.

D. Attendance APIs

Method	Endpoint	Description
GET	/attendance	Retrieve attendance data
GET	/attendance/{id}	Retrieve attendance of 1 employee

Source: **payroll.attendance**

E. Reporting APIs

Method	Endpoint	Description
GET	/reports/hr	HR reports from HUMAN_2025
GET	/reports/payroll	Payroll summaries
GET	/reports/attendance	Attendance summaries
GET	/reports/dividends	Dividend history

These reports require joining **HR + Payroll** via EmployeeID.

F. Alerts APIs

Method	Endpoint	Description
GET	/alerts	Generate all system alerts

Alerts include:

- Work anniversaries
- Excessive leave
- Salary anomalies

3.2 Data Synchronization & Transaction Handling

Every create/update operation involving both databases must:

- Start a transaction
- Write to SQL Server (HUMAN_2025)
- Write to MySQL (payroll)
- Commit only when both succeed
- Rollback both if one fails

Example Sync Flow

POST /employees

1. Insert into SQL Server → HUMAN_2025.Employees
2. Insert into MySQL → employees_payroll
3. If MySQL fails → rollback SQL Server
4. Return 201 + synced employee

Block update if:

- DepartmentID does not exist in HR DB
- PositionID does not exist in HR DB
- Duplicated EmployeeID in payroll side

Block deletion if:

- EmployeeID exists in salaries (Payroll)
- EmployeeID exists in Dividends (HR)

3.3 Validation & Error Handling

The backend must enforce validation rules derived from:

- DB schema constraints
- Business rules in CS1
- CEO Memo requirements

Validate:

- Required fields (FullName, HireDate, DepartmentID…)
- Field lengths
- Valid foreign keys
- Duplicates in payroll DB

Standardized HTTP Responses

Code	Meaning
200	Success
201	Created
400	Invalid input
404	Not found
409	Conflict (duplicate, constraint)
500	Database/transaction error

3.4 Unit Testing & Integration Testing

Students must implement:

Unit Testing

Frameworks:

- xUnit / NUnit (C#)
- pytest / unittest (Python)

Tests required:

- DB connection tests
- CRUD function tests
- Sync logic tests
- Validation rule tests

API Testing

Using:

- Postman
- Swagger
- Insomnia

Test cases must include:

- Add employee → appears in both DBs
- Attempt duplicate insert → 409 Conflict
- Delete employee with salary → blocked
- Update department/position → synced
- Payroll retrieval accuracy
- Alerts correctness

Evidence required:

- Screenshots
- JSON responses
- Database screenshots

3.5 Documentation Requirements

Students must deliver **five documents**, continuing the numbering from CS2:

Document 6 – API Implementation Document (NEW)

- Endpoint list
- Request/response examples
- Validation rules
- Error codes
- Rollback procedures

Document 7 – Test Plan (NEW)

- Test scope
- Tools
- Approach (unit vs integration)
- Test environment

Document 8 – Test Case Document (NEW)

A detailed table per test case:

- Test ID
- API
- Input
- Expected Output
- Actual Output
- Status

Document 9 – Updated SRS v2 (UPDATED)

Includes:

- Technical constraints
- API architecture
- Sync flow diagrams

Document 10 – Updated Data Mapping Document (UPDATED)

Add mapping for fields **used in actual API code**, including:

- Insert fields
- Update fields
- Validation fields

4. Software Deliverables

Required API functionality:

- **Employee Management**
- **Department & Position Sync**
- **Payroll Retrieval**
- **Attendance Retrieval**
- **Reporting**
- **Alerts**
- **Full Sync Logic**

5. Evaluation Criteria

Rubric includes:

- API correctness
- Sync reliability
- Transaction safety
- Error handling
- Test quality
- Coding style
- Documentation completeness